TECHNICAL INTEGRATION PAPER

Sakshi on Arkiv

Entity model, lifetime semantics, write and read paths, and failure modes

Version 0.1

October 2026

Network: Arkiv Tiramisu testnet, chain ID 7738577

SDK: @arkiv-network/sdk 0.8.1

Repository: github.com/Rajshah1302/sakshi

> This paper documents how Sakshi uses Arkiv at the entity, attribute, lifetime, and query level. It covers the three entity types, the exact attribute schemas and value types, the check-in transaction, the read and expiry-detection paths, the ownership model, the SDK limits that shaped the design, and the failure modes observed while building. It distinguishes what is implemented from what is specified but not yet implemented. It deliberately excludes product narrative, market context, and pricing.

Contents

1. Scope and environment
2. Why Arkiv: the properties the design depends on
3. Entity types
4. Attribute schema and value types
5. Payload encoding
6. Lifetime model: Entity Expiration and Lifetime Extension
7. Write path: creation
8. Write path: check-in
9. Read path and exact queries
10. Lapse detection without an expiry event
11. Release path
12. Ownership, authorship, and the creator field
13. SDK limits and constraints
14. Error taxonomy and handling
15. Cryptographic envelope
16. Implemented versus specified
17. Friction log, technical edition
18. Reproduction
19. File map
20. Appendix A: exact raw RPC queries
21. Appendix B: references

1. Scope and environment

Sakshi stores a sealed evidence payload and a time-scoped heartbeat. The technical surface is a set of Arkiv entities plus a local cryptographic envelope. This paper concerns only that surface.

Item	Value
Network	Arkiv Tiramisu testnet
Chain ID	`7738577` (`0x7614d1`)
HTTP RPC	`https://rpc.tiramisu.db-chain.testnet.arkiv.network`
WebSocket RPC	`wss://rpc.tiramisu.db-chain.testnet.arkiv.network`
Block time	2 seconds
SDK	`@arkiv-network/sdk` 0.8.1
Client	`viem` 2.x, `http()` transport
Gas token	GLM
Namespacing	`project = "sakshi"` on every entity

2. Why Arkiv: the properties the design depends on

The design uses four properties that are native to Arkiv and awkward to reproduce elsewhere.

1. **Per-entity expiration.** Every entity is created with a lifetime. When the lifetime ends, the entity leaves the query surface. A stored timestamp is inert data; an expiring entity is a state change performed by the database.
2. **Lifetime Extension.** `extendEntity` moves an expiry later, which gives the check-in a native renewal operation.
3. **Immutable creator, mutable owner.** `$creator` is fixed at creation and is the authorship anchor. `$owner` controls writes, extension, transfer, and deletion.
4. **Public reads without an operator backend.** Any client with an RPC endpoint can read the entities. The record does not depend on the application server.

The design deliberately keeps the plaintext, the content key, and the shares off Arkiv.

3. Entity types

Entity types are modelled as a `type` attribute. Every entity also carries `project = "sakshi"`.

Type	Owner / creator	Creation flags	Expiration	Role
`vault`	holder	`readonly: true`	`ExpirationTime.permanent()`	Durable anchor; holds ciphertext and hash.
`beat`	holder	none	`ExpirationTime.atDate(deadline)`	The switch; the only expiring entity.
`release`	watchdog	`readonly: true`	`ExpirationTime.permanent()`	Records that the switch fired.

The split is intentional. If the vault expired with the beat, the switch would delete the evidence at the moment of release. The vault must outlive the switch, so it is permanent. The beat must vanish when the holder stops checking in, so its lifetime is the deadline.

4. Attribute schema and value types

Arkiv attributes are typed. The type determines which query operators the index supports. Numeric types (`u64`, `i32`, `u256`, `dec`) support range comparisons; `str` supports equality and prefix; `bool`, `addr`, `key`, and `bytes32` support equality only.

4.1 Vault

Attribute	Type	Example	Query use
`project`	str	`sakshi`	namespace equality
`type`	str	`vault`	type equality
`evidence_hash`	bytes32	`0x...`	equality only; integrity reference
`created_ms`	u64	`1760000000000`	range, if needed

4.2 Beat

Attribute	Type	Example	Query use
`project`	str	`sakshi`	namespace equality
`type`	str	`beat`	type equality
`vault_key`	key	`0x...`	equality; joins beat to vault
`deadline`	u64	`1760000600000`	range (expiring soon, lapsed)
`interval_s`	u64	`604800`	display; renewal input

4.3 Release

Attribute	Type	Example	Query use
`project`	str	`sakshi`	namespace equality
`type`	str	`release`	type equality
`vault_key`	key	`0x...`	equality; release for a vault
`released_ms`	u64	`1760000700000`	range

> **Naming rule.** Arkiv attribute names are 1 to 32 ASCII characters: a lowercase letter first, then lowercase letters, digits, `.`, `-`, and `_`. Uppercase letters are rejected at the node. The design uses snake_case (`valid_until`, `interval_s`) for this reason.

5. Payload encoding

The vault payload is JSON, stored as bytes with content type `application/json`:

```
{ "v": 1, "nonce": "<base64>", "ciphertext": "<base64>" }
```

The ciphertext is AES-256-GCM output with the 16-byte authentication tag appended. The payload is capped at 128 KiB by Arkiv, so only small evidence or a pointer to off-chain evidence fits.

The beat payload is an empty `text/plain` body. The release payload is empty as well.

The payload is never queried. Everything the application filters on is an attribute.

6. Lifetime model: Entity Expiration and Lifetime Extension

6.1 Expiration

Every entity is created with an expiry. The wire form is a pair: an absolute target block and a minimum lifetime in blocks. The engine resolves both against the block the transaction lands in and takes the later value.

```
ExpirationTime.permanent()                // vault, release
ExpirationTime.atDate(new Date(deadline))  // beat
ExpirationTime.fromDays(7)                // relative lifetime
ExpirationTime.atBlock(1_200_000n)        // absolute block
```

Durations must be a whole number of 2-second blocks. `fromSeconds(7)` is rejected; `fromSeconds(8)` is accepted. Deadlines derived from wall-clock timestamps should use `atDate`.

6.2 Extension

`extendEntity` sets a new expiry. It is not additive: `fromDays(30)` means 30 days from now, not 30 days after the current expiry. An extension to an earlier block is rejected. A later expiry moves the entity out; an equal expiry is a no-op.

```
const result = await wallet.extendEntity({
  entityKey: beatKey,
  expires: ExpirationTime.atDate(new Date(deadlineMs)),
});
```

6.3 Expiration has no event

When an entity expires it leaves the query surface and every operation against it is rejected. Arkiv does not emit an event for expiration, and there is no grace period. The application must therefore track the deadline or poll `$expiresAt`.

7. Write path: creation

The current implementation performs two writes: the vault, then the beat.

```
const keyBytes = generateKey();
const sealed = encrypt(keyBytes, evidence);
const evidenceHash = keccak256(toBytes(JSON.stringify(sealed)));

const vault = await wallet.createEntity({
  payload: jsonToPayload({ v: 1, ...sealed }),
  contentType: "application/json",
  attributes: {
    project: str("sakshi"),
    type: str("vault"),
    evidence_hash: bytes32(evidenceHash),
    created_ms: u64(Date.now()),
  },
  expires: ExpirationTime.permanent(),
  flags: { readonly: true },
});

const shares = split(keyBytes, 3, 2);
const beat = await wallet.createEntity({
  payload: stringToPayload(""),
  contentType: "text/plain",
  attributes: {
    project: str("sakshi"),
    type: str("beat"),
    vault_key: key(vault.entityKey),
    deadline: u64(deadlineMs),
    interval_s: u64(intervalSeconds),
  },
  expires: ExpirationTime.atDate(new Date(deadlineMs)),
});
```

Two technical points:

- **Ordering is required.** The beat references the vault key, so the vault must exist before the beat is written. The current code does this with two transactions. A single atomic batch would require `predictEntityKeys` plus a known salt.
- **Hash mismatch bug.** The hash is computed over `JSON.stringify(sealed)` but the payload stores `{ v: 1, ...sealed }`. The stored hash therefore does not cover the exact payload bytes. This is a correctness bug, not an Arkiv limitation.

8. Write path: check-in

A check-in must update two things: the `deadline` attribute (read by the UI and the watchdog) and the entity expiry (enforced by the network). The implementation uses one atomic batch.

```
const deadlineMs = Date.now() + intervalS * 1000;

await wallet.executeBatch({
  patches: [{
    entityKey: beatKey,
    set: { deadline: u64(deadlineMs) },
  }],
  extensions: [{
    entityKey: beatKey,
    expires: ExpirationTime.atDate(new Date(deadlineMs)),
  }],
});
```

The SDK applies batch operations in a fixed order: creates, patches, deletes, extensions, ownership changes. A later operation sees the effects of an earlier one. The patch therefore lands before the extension.

Only the owner can patch and extend the beat. The beat is deliberately created without `readonly`, because `readonly` freezes the payload and attributes even for the owner; a check-in would be impossible.

9. Read path and exact queries

Reads use a public client. A query is a boolean expression; there is no `SELECT` or `WHERE` keyword in the raw form.

```
// live beats in the namespace
client.select("*")
  .where(and(eq("project", "sakshi"), eq("type", "beat")))
  .limit(200)
  .fetch();

// releases for a vault
client.select("*")
  .where(and(
    eq("project", "sakshi"),
    eq("type", "release"),
    eq("vault_key", key(vaultKey)),
  ))
  .fetch();

// one entity by key
client.getEntity(vaultKey);
```

Properties that shaped the read path:

- **No server-side ordering.** There is no `orderBy`. A time-ordered list must be sorted client-side or fetched through numeric range predicates on `deadline` or `created_ms`.
- **200 results per page.** `MAX_LIMIT` is 200. Pagination uses a cursor bound to a block; `hasNextPage()` and `next()` walk the pages. The current code does not yet paginate.
- **The SDK requires a predicate.** The builder rejects a match-all query, so every query starts with `project = "sakshi"`.
- **Typed literals.** A bare number defaults to `i32`. Comparing a `u64` attribute with a bare number can silently return zero rows; the literal must be wrapped with `u64()`.

10. Lapse detection without an expiry event

Because expiration emits no event, the switch cannot be observed by subscribing to a stream. The watchdog uses a poll plus deadline tracking:

```
try {
  const beat = await getBeat(beatKey);       // getEntity under the hood
  vaultKey = beat.vaultKey;
  if (Date.now() >= beat.deadline) {
    await writeRelease(vaultKey);
  }
} catch (error) {
  if (error instanceof NoEntityFoundError) {
    await writeRelease(vaultKey);            // beat has expired
  } else {
    throw error;                             // network error: do not release
  }
}
```

The distinction between `NoEntityFoundError` and a transport error is critical. A transient RPC failure is not a lapse. The current UI does not make this distinction correctly, and it can write a false release; the fix is to keep the last known state, show an unreachable banner, and only fire on a confirmed expiry.

`getEntity` throws the same `NoEntityFoundError` for an entity that expired, was deleted, or never existed. A production system should add a tombstone or status entity if it needs to distinguish these cases.

Historical verification is possible with `.atBlock(block)`, which returns entities as they existed at a past block, including entities that have since expired.

11. Release path

```
const release = await wallet.createEntity({
  payload: stringToPayload(""),
  contentType: "text/plain",
  attributes: {
    project: str("sakshi"),
    type: str("release"),
    vault_key: key(vaultKey),
    released_ms: u64(Date.now()),
  },
  expires: ExpirationTime.permanent(),
  flags: { readonly: true },
});
```

The release is written by the watchdog wallet, so its `$creator` is the watchdog. The current build does not authenticate the release endpoint and does not gate unseal on the existence of a release. Both are specified, not implemented.

12. Ownership, authorship, and the creator field

System field	Mutability	Meaning in Sakshi
`$creator`	immutable	who wrote the entity; the authorship anchor
`$owner`	mutable	who controls writes, extension, transfer, deletion
`$createdAt`	immutable	block height of creation
`$expiresAt`	mutable	current expiry block

The designer can query these fields but cannot set them. `changeOwnership` transfers control but does not change `$creator`; a `createdBy()` filter still finds the entity.

> **Current gap.** The application does not yet filter by `createdBy()`. Reads use `getEntity` by key and `select` on `project` and `type`. A production build must bind trust to `$creator` and reject entities from unexpected wallets, especially for the release.

13. SDK limits and constraints

Constraint	Value	Consequence
Attribute count	30 cells (32 with payload and content type)	keep schemas narrow
Payload size	128 KiB	store a pointer for large evidence
String attribute	128 UTF-8 bytes	long text goes in the payload
Attribute name	1–32 ASCII, lowercase first	snake_case names
Result page	200	cursor pagination
Block time	2 s	deadlines and durations in blocks
Ordering	none	client-side sort
Expiry event	none	poll or track deadlines
Read budget	keyless quota	access key server-side; proxy for public reads

14. Error taxonomy and handling

Error	Cause	Handling
`InvalidExpiryError`	lifetime not a whole block multiple, or dead on arrival	round to blocks; use `atDate` for deadlines
`InvalidAttributeNameError`	uppercase or reserved name	lowercase snake_case
`TooManyAttributesError`	more than the cell limit	move fields to the payload
`UnknownAttributeTypeError`	missing or wrong typed value	wrap every value in its constructor
`NoEntityFoundError`	expired, deleted, or never existed	treat as lapsed only after confirming the deadline
`EntityMutationError`	node rejected the write	surface the node message; retry only on transport errors
`ConflictingMutationError`	concurrent writes to the same entity	serialize check-ins per beat

15. Cryptographic envelope

The Arkiv layer stores an opaque envelope. The envelope is produced locally.

Property	Implementation
Content encryption	AES-256-GCM, 12-byte nonce, 16-byte tag appended
Key	32 random bytes, generated per vault
Key splitting	Shamir over GF(2^8), polynomial 0x1b, 3 shares, threshold 2
Share encoding	hex of `[x, y0, y1, ...]`
Hash	keccak256 or SHA-256 over the sealed payload; see the hash bug in section 7

The cryptographic envelope is outside Arkiv's trust boundary. Arkiv enforces existence, lifetime, and authorship; it does not verify the ciphertext or the claim. Decryption is a client operation.

16. Implemented versus specified

Capability	Status
Typed attributes, `project` / `type` namespaces	implemented
Per-entity expiration (vault permanent, beat expiring)	implemented
`extendEntity` check-in	implemented
Atomic `executeBatch` patch plus extension	implemented
`readonly` vault and release	implemented
`key` reference from beat to vault	implemented
Release entity on lapse	implemented
Public read without the application backend (CLI)	implemented
`createdBy()` trust filtering	not implemented
Cursor pagination for lists	not implemented
`watchEntityEvents` over WebSocket	not implemented
Atomic batch creation with `predictEntityKeys`	not implemented
Release-gated unseal	not implemented
On-chain entities from a funded run	not yet created

17. Friction log, technical edition

1. **Expiry is silent.** No event, no grace period, and expired entities disappear from queries. A switch must poll `$expiresAt` or track deadlines.
2. **`readonly` does not stop extension, transfer, or deletion.** It freezes attributes and payload only. Control therefore comes from ownership, not the flag.
3. **Durations must be whole blocks.** `fromSeconds(7)` is rejected; use `atDate` for wall-clock deadlines.

4. `executeBatch` **order is fixed** (creates, patches, deletes, extensions, ownership), which the atomic check-in relies on.
5. **Attribute read-back is a tagged cell**, so numeric values must be parsed.
6. **No server-side ordering and a 200-row page.** Lists must be paged and sorted client-side.
7. **The SDK requires a predicate** where raw RPC accepts `*`.
8. **Keyless read budget is small**, and an access key in browser code leaks. Public reads need a server-side proxy or a deliberate keyless budget.

18. Reproduction

```
git clone https://github.com/Rajshah1302/sakshi
cd sakshi
npm install
npm run simulate         # AES-GCM + Shamir
npm run simulate:switch  # create, check-in, lapse, release, unseal
npm run test:backend     # demo backend end to end
npm run typecheck:all
npm run build:web
```

For the live Arkiv path, a funded Tiramisu wallet is required:

```
npm run wallet -- --write
npm run create -- ./evidence.txt 604800
npm run checkin -- <beatKey> 604800
npm run watch -- <beatKey> <vaultKey>
npm run release -- <vaultKey> <shareA> <shareB> released.bin
```

19. File map

Path	Responsibility
`src/arkiv/client.ts`	public and wallet clients
`src/arkiv/entities.ts`	constants and attribute readers
`src/arkiv/mutations.ts`	create, check-in, release
`src/arkiv/queries.ts`	get, list, unseal
`src/crypto/aes.ts`	AES-256-GCM
`src/crypto/shamir.ts`	GF(2^8) secret sharing
`server/backend.ts`	live and demo backends
`server/index.ts`	Express API
`web/src/browser-crypto.ts`	WebCrypto and browser Shamir
`web/src/demo.ts`	client-side demo backend
`arkiv/schema.md`	entity contract and exact queries
`arkiv/friction.md`	the full friction log

20. Appendix A: exact raw RPC queries

```
project = str('sakshi') AND type = str('beat')

project = str('sakshi') AND type = str('release') AND vault_key = key('0x…')

$creator = addr(0x…) AND project = str('sakshi')

app = str('sakshi') AND type = str('beat') AND deadline > u64(1760000000000)
```

The raw grammar uses typed literals: `str('…')`, `u64(…)`, `key(0x…)`, `addr(0x…)`. Boolean operators are `AND`, `OR`, `NOT`, and there is no `!=`; use `NOT (attr = value)`.

21. Appendix B: references

1. Arkiv documentation, https://docs.arkiv.network
2. Arkiv Tiramisu network, https://docs.arkiv.network/networks/tiramisu/
3. TypeScript SDK 0.8, querying and mutating data, https://docs.arkiv.network/typescript-sdk/
4. Shamir, A., How to share a secret, 1979.
5. NIST SP 800-38D, Galois/Counter Mode.

End of paper.